MOCKITO EXCERCISES

Exercise 1: Mocking and Stubbing

ExternalApi.java

```
/**
 * ExternalApi — Interface representing a third-party/external API.
 * In production, this would make real HTTP/DB calls.
 * In tests, we MOCK this to avoid real network dependency.
 */
public interface ExternalApi {

    // Fetches raw data from external source
    String getData();

    // Fetches data for a specific user ID
    String getUserData(int userId);

    // Checks if the external service is available
    boolean isServiceAvailable();
}
```

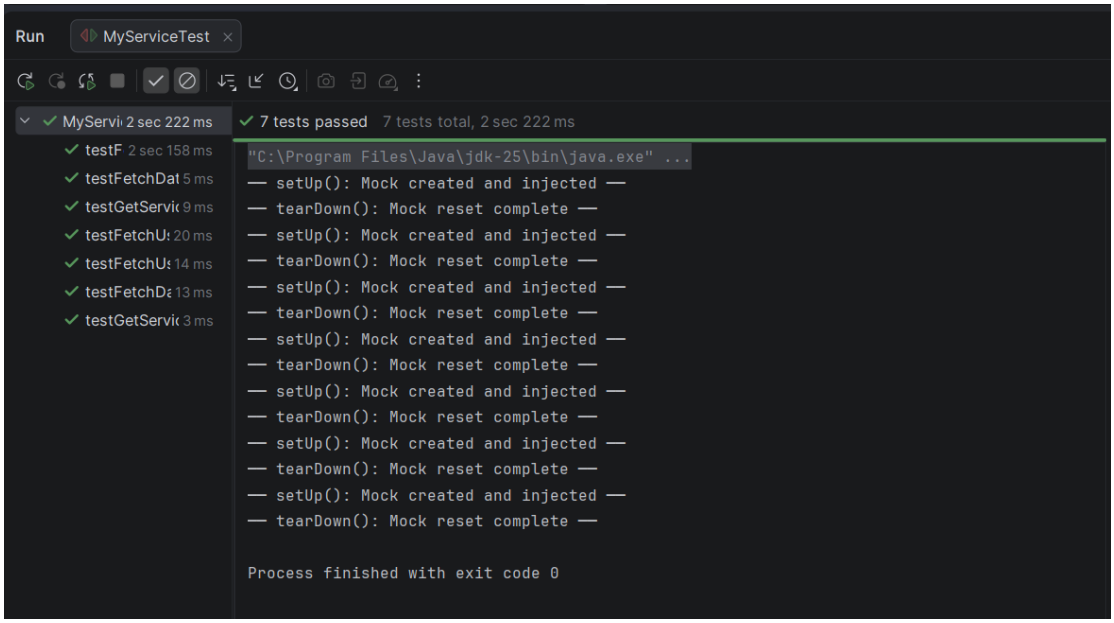
ExternalApiImpl.java

```
/**
 * ExternalApiImpl — Real implementation of ExternalApi.
 * Would make actual API calls in production.
 * NOT used in unit tests — replaced by Mockito mock.
 */
public class ExternalApiImpl implements ExternalApi {

    @Override
    public String getData() {
        // Simulates real API call (e.g., HTTP GET)
        return "Real API Data";
    }

    @Override
    public String getUserData(int userId) {
        return "Real User Data for ID: " + userId;
    }

    @Override
    public boolean isServiceAvailable() {
        return true; // Would ping real endpoint
    }
}
```

MyService.java

```
/**
 * MyService — Business logic layer.
 * Depends on ExternalApi (injected via constructor).
 * This is the class we actually want to test.
 *
 * Uses Dependency Injection — makes it fully mockable.
 */
public class MyService {

    private final ExternalApi externalApi;

    // Constructor Injection — best practice for testability
    public MyService(ExternalApi externalApi) {
        this.externalApi = externalApi;
    }

    // Fetches and processes general data from API
    public String fetchData() {
        String data = externalApi.getData();
        if (data == null || data.isEmpty()) {
            return "No Data Available";
        }
        return data;
    }

    // Fetches data for a specific user
    public String fetchUserData(int userId) {
        if (userId <= 0) {
            return "Invalid User ID";
        }
        return externalApi.getUserData(userId);
    }

    // Returns service status message
    public String getServiceStatus() {
        boolean available = externalApi.isServiceAvailable();
        return available ? "Service is UP" : "Service is DOWN";
    }
}
```

OUTPUT:



Exercise 2: VERIFYING INTERACTIONS

ExternalApi.java

```
/**
 * ExternalApi — Interface representing a third-party/external API.
 * In production, this would make real HTTP/DB calls.
 * In tests, we MOCK this to avoid real network dependency.
 */
public interface ExternalApi {

    // Fetches raw data from external source
    String getData();

    // Fetches data for a specific user ID
    String getUserData(int userId);

    // Checks if the external service is available
    boolean isServiceAvailable();
}
```

MyService.java

```
/**
 * MyService — Business logic layer.
 * Depends on ExternalApi (injected via constructor).
 * This is the class we actually want to test.
 *
 * Uses Dependency Injection — makes it fully mockable.
 */
public class MyService {
```

```
    private final ExternalApi externalApi;

    // Constructor Injection — best practice for testability
    public MyService(ExternalApi externalApi) {
        this.externalApi = externalApi;
    }

    // Fetches and processes general data from API
    public String fetchData() {
        String data = externalApi.getData();
        if (data == null || data.isEmpty()) {
            return "No Data Available";
        }
        return data;
    }

    // Fetches data for a specific user
    public String fetchUserData(int userId) {
        if (userId <= 0) {
            return "Invalid User ID";
        }
        return externalApi.getUserData(userId);
    }

    // Returns service status message
    public String getServiceStatus() {
        boolean available = externalApi.isServiceAvailable();
        return available ? "Service is UP" : "Service is DOWN";
    }
}
```

OrderService.java

```
/**
 * OrderService — Simulates a real order processing system.
 * Depends on ExternalApi for payment and notification calls.
 * Used to demonstrate advanced Mockito interaction verification.
 */
public class OrderService {

    private final ExternalApi externalApi;

    public OrderService(ExternalApi externalApi) {
        this.externalApi = externalApi;
    }

    // Places an order for a specific user
    public String placeOrder(int userId, String item) {
        if (userId <= 0 || item == null || item.isEmpty()) {
            return "Invalid Order";
        }
```

```
        String userData = externalApi.getUserData(userId);
        String confirmation = externalApi.getData();


    return "Order placed for " + userData + " | Item: " + item
                + " | Ref: " + confirmation;
    }

    // Checks availability before ordering
    public String checkAndOrder(int userId, String item) {
        boolean available = externalApi.isServiceAvailable();
        if (!available) {
            return "Service Unavailable — Order Cancelled";
        }
        return placeOrder(userId, item);
    }

    // Bulk order — calls getUserData multiple times
    public void placeBulkOrders(int userId, String[] items) {
        for (String item : items) {
            externalApi.getUserData(userId);
            externalApi.getData();
        }
    }
}
```

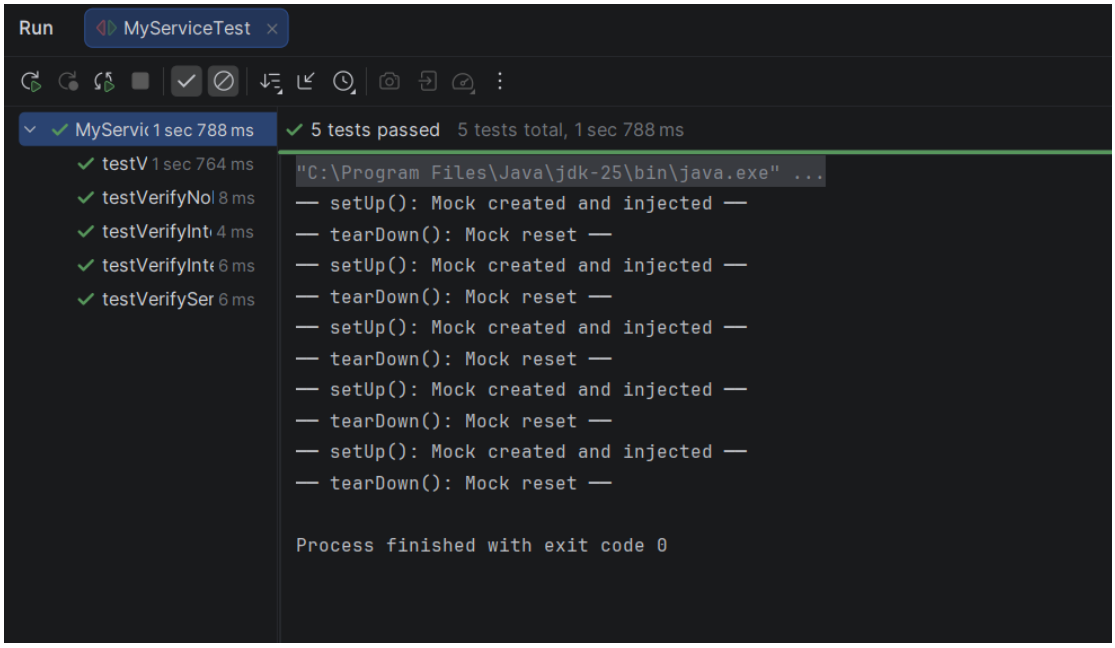
MyServiceTest.java

```
import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import org.mockito.Mockito;

import static org.junit.Assert.*;
import static org.mockito.Mockito.*;

/**
 * MyServiceTest — Extended with Interaction Verification.
 *
 *  ✓ verify() — method was called
 *  ✓ verify(times(n)) — called exact number of times
 *  ✓ verify(never()) — method was never called
 *  ✓ verify(atLeast/atMost) — call frequency bounds
 *  ✓ verifyNoMoreInteractions() — no unexpected calls
 *  ✓ Argument matching with eq() and anyInt()
 */
public class MyServiceTest {

    private ExternalApi mockApi;
    private MyService service;

    @Before
    public void setUp() {
        mockApi  = Mockito.mock(ExternalApi.class);
```

```
        service  = new MyService(mockApi);
        System.out.println("── setUp(): Mock created and injected ──");
    }




@After
    public void tearDown() {
        Mockito.reset(mockApi);
        System.out.println("── tearDown(): Mock reset ──");
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 1: Basic verify — getData() was called once
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testVerifyInteraction_GetDataCalledOnce() {

        // ARRANGE
        when(mockApi.getData()).thenReturn("Mock Data");

        // ACT
        service.fetchData();

        // ASSERT + VERIFY
        verify(mockApi).getData();                  // called at least once
        verify(mockApi, times(1)).getData();        // called exactly once
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 2: Verify with specific argument — eq()
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testVerifyInteraction_WithSpecificArgument() {

        // ARRANGE
        when(mockApi.getUserData(101)).thenReturn("Shree Rithi");

        // ACT
        service.fetchUserData(101);

        // ASSERT + VERIFY — must be called with EXACTLY 101
        verify(mockApi, times(1)).getUserData(eq(101));
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 3: Verify never called — invalid input
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testVerifyInteraction_NeverCalled_InvalidId() {

        // ARRANGE — no stub needed
```

```
        // ACT
        service.fetchUserData(-5);

        // ASSERT + VERIFY — API must NEVER be called for invalid ID
        verify(mockApi, never()).getUserData(anyInt());
        verify(mockApi, never()).getData();
    }




// ═══════════════════════════════════════════════════════════════
    //  TEST 4: Verify no more interactions after fetchData
    // ═══════════════════════════════════════════════════════════════
    @Test
    public void testVerifyNoMoreInteractions() {

        // ARRANGE
        when(mockApi.getData()).thenReturn("Clean Data");

        // ACT
        service.fetchData();

        // ASSERT + VERIFY
        verify(mockApi).getData();

        // Ensures no OTHER unexpected methods were called on mock
        verifyNoMoreInteractions(mockApi);
    }

    // ═══════════════════════════════════════════════════════════════
    //  TEST 5: Verify isServiceAvailable() interaction
    // ═══════════════════════════════════════════════════════════════
    @Test
    public void testVerifyServiceStatusCheck() {

        // ARRANGE
        when(mockApi.isServiceAvailable()).thenReturn(true);

        // ACT
        service.getServiceStatus();

        // ASSERT + VERIFY
        verify(mockApi, times(1)).isServiceAvailable();
        verify(mockApi, never()).getData();          // getData must NOT be triggered
        verify(mockApi, never()).getUserData(anyInt());
    }
}
```

OUTPUT:



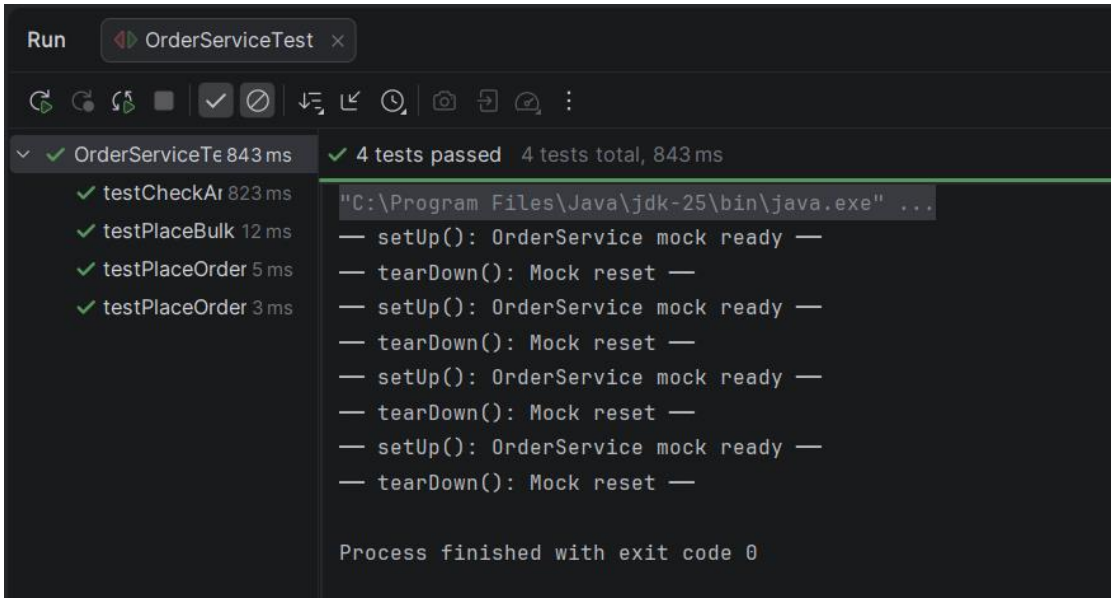
OrderServiceTest.java

```
import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import org.mockito.Mockito;

import static org.junit.Assert.*;
import static org.mockito.Mockito.*;

/**
 * OrderServiceTest — Advanced Mockito Interaction Verification.
 *
 *  ✓ verify with exact argument matching
 *  ✓ atLeast() / atMost() frequency verification
 *  ✓ Interaction order doesn't matter (default Mockito)
 *  ✓ Bulk interaction verification
 *  ✓ Short-circuit verification (service down = no order placed)
 */
public class OrderServiceTest {

    private ExternalApi mockApi;
    private OrderService orderService;

    @Before
    public void setUp() {
        mockApi      = Mockito.mock(ExternalApi.class);
        orderService = new OrderService(mockApi);
        System.out.println("── setUp(): OrderService mock ready ──");
```

```
    }



@After
    public void tearDown() {
        Mockito.reset(mockApi);
        System.out.println("── tearDown(): Mock reset ──");
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 1: placeOrder calls getUserData + getData
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testPlaceOrder_VerifiesBothApiCalls() {

        // ARRANGE
        when(mockApi.getUserData(201)).thenReturn("Shree Rithi");
        when(mockApi.getData()).thenReturn("ORD-9981");

        // ACT
        String result = orderService.placeOrder(201, "Laptop");

        // ASSERT
        assertTrue("Result must contain user name",
                result.contains("Shree Rithi"));
        assertTrue("Result must contain item",
                result.contains("Laptop"));

        // VERIFY — both API calls must happen exactly once
        verify(mockApi, times(1)).getUserData(eq(201));
        verify(mockApi, times(1)).getData();
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 2: Invalid order — no API interactions
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testPlaceOrder_InvalidInput_NoApiCalls() {

        // ARRANGE — invalid userId

        // ACT
        String result = orderService.placeOrder(-1, "Phone");

        // ASSERT
        assertEquals("Invalid order must return error",
                "Invalid Order", result);

        // VERIFY — zero interactions with mock
        verify(mockApi, never()).getUserData(anyInt());
        verify(mockApi, never()).getData();
        verifyNoMoreInteractions(mockApi);
    }
```

```
    // ═══════════════════════════════════════════════════════════
    //  TEST 3: Service down — only isServiceAvailable called
    // ═══════════════════════════════════════════════════════════
    @Test



public void testCheckAndOrder_ServiceDown_ShortCircuits() {

        // ARRANGE
        when(mockApi.isServiceAvailable()).thenReturn(false);

        // ACT
        String result = orderService.checkAndOrder(301, "Tablet");

        // ASSERT
        assertEquals("Cancelled message must match",
                "Service Unavailable — Order Cancelled", result);

        // VERIFY — only availability checked, order never placed
        verify(mockApi, times(1)).isServiceAvailable();
        verify(mockApi, never()).getUserData(anyInt());
        verify(mockApi, never()).getData();
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 4: Bulk orders — atLeast / atMost verification
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testPlaceBulkOrders_VerifyCallFrequency() {

        // ARRANGE
        String[] items = {"Phone", "Tablet", "Laptop"};
        when(mockApi.getUserData(401)).thenReturn("Bulk User");
        when(mockApi.getData()).thenReturn("BULK-REF");

        // ACT
        orderService.placeBulkOrders(401, items);

        // VERIFY — called exactly 3 times (once per item)
        verify(mockApi, times(3)).getUserData(eq(401));
        verify(mockApi, times(3)).getData();

        // VERIFY — frequency bounds
        verify(mockApi, atLeast(2)).getUserData(anyInt());
        verify(mockApi, atMost(5)).getData();
    }
}
```

OUTPUT :


Run
OrderServiceTest
OrderServiceTe 843 ms
testCheckAr 823 ms
testPlaceBulk 12 ms
testPlaceOrder 5 ms
testPlaceOrder 3 ms
4 tests passed 4 tests total, 843 ms
"C:\Program Files\Java\jdk-25\bin\java.exe" ...
— setUp(): OrderService mock ready —
— tearDown(): Mock reset —
— setUp(): OrderService mock ready —
— tearDown(): Mock reset —
— setUp(): OrderService mock ready —
— tearDown(): Mock reset —
— setUp(): OrderService mock ready —
— tearDown(): Mock reset —
Process finished with exit code 0